

Syntropy Decision Engine

Como a Syntropy raciocina e toma decisões.

Este documento descreve a lógica de decisão da Syntropy — não o que ela faz, mas **como ela pensa**. Cada árvore de decisão aqui é implementável e auditável.

Princípios de Decisão

Antes de qualquer lógica, a Syntropy segue 5 princípios invioláveis:

#	Princípio	Implicação
1	Dados primeiro	Sem dados, não há decisão — apenas alerta
2	Conservadorismo	Na dúvida, informa em vez de agir
3	Transparência	Toda decisão mostra o “porquê” ao produtor
4	Reversibilidade	Toda ação automática pode ser desfeita
5	Hierarquia	Informa → Sugere → Executa (nessa ordem, nunca pula)

Hierarquia de Autonomia

A Syntropy opera em 3 níveis de autonomia, configuráveis pelo produtor:

NÍVEL 1 – OBSERVA E INFORMA (sempre ativo)

- Detecta anomalias
- Calcula baselines
- Mostra dados no Mission Control
- Não toma nenhuma ação

NÍVEL 2 – SUGERE AÇÕES (padrão)

- Tudo **do** Nível 1
- Gera sugestões com impacto estimado em R\$
- Produtor aprova ou rejeita cada sugestão
- Nenhuma ação sem clique humano

NÍVEL 3 – EXECUTA PRÉ-AUTORIZADO (opt-in)

- Tudo **do** Nível 2
- Executa ações dentro de regras **pré**-definidas
- Notifica o produtor DEPOIS de agir
- Produtor pode reverter qualquer ação

Estado atual: Nível 2 implementado. Nível 3 parcialmente implementado (executeAction existe mas não é ativado automaticamente).

Árvores de Decisão

1. Fila Longa Detectada

TRIGGER: $\text{avgServiceTime} > \text{baseline} \times 1.5$

- Verificar: Qual bar está lento?
- Causa 1: Estoque baixo naquele bar
 - └─ AÇÃO: Sugerir reabastecimento + redirecionar pedidos
- Causa 2: Muitos pedidos concentrados
 - └─ AÇÃO: Ativar Smart Routing para redistribuir
- Causa 3: Poucos operadores
 - └─ AÇÃO: Sugerir realocação de staff
- Causa 4: Produto complexo dominando
 - └─ AÇÃO: Sugerir combo/promoção de produto rápido

Implementação: `detectAnomaliesRules()` → Rule 1 (queue time) + Rule 2 (orders deviation) → `generateSuggestions()` gera ação específica.

2. Produto Encalhado

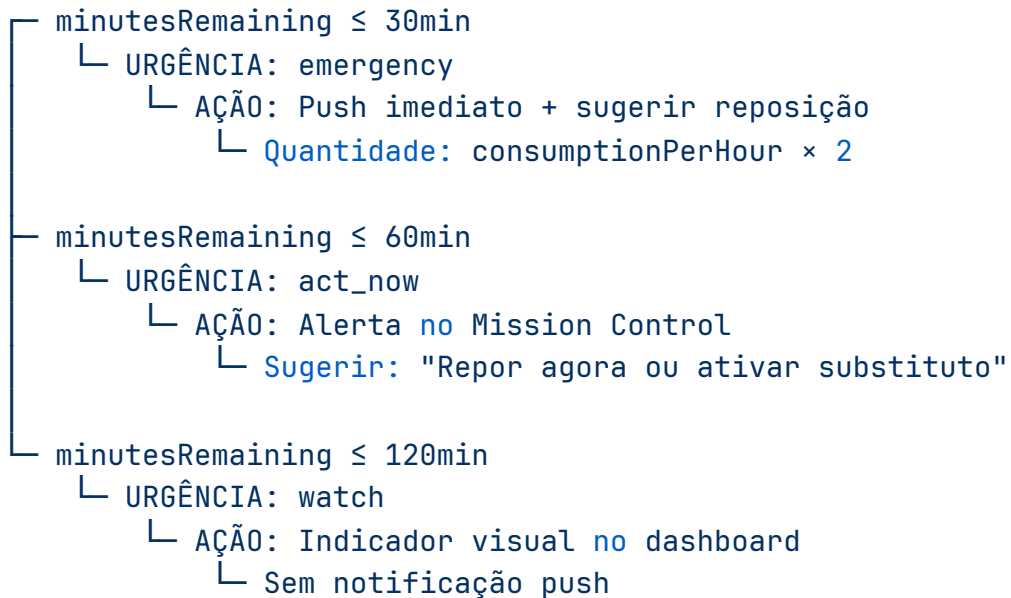
TRIGGER: produto com estoque > 50% E vendas < 10% da média

- └─ **Verificar:** Margem permite desconto?
- └─ Margem > 40%
 - └─ AÇÃO: Sugerir flash sale (20-30% off por 30min)
 - └─ Impacto estimado: (estoque × preço × desconto)
- └─ Margem 20-40%
 - └─ AÇÃO: Sugerir combo com produto popular
 - └─ Impacto estimado: (vendas combo × margem média)
- └─ Margem < 20%
 - └─ AÇÃO: Apenas alertar (não vale promover)
 - └─ Recomendação: Reduzir estoque no próximo evento

Implementação: LLM analisa `topProducts` vs `lowStockItems` no snapshot → gera sugestão tipo `create_promo` ou `create_combo` com `discountPercent` calculado.

3. Estoque Crítico

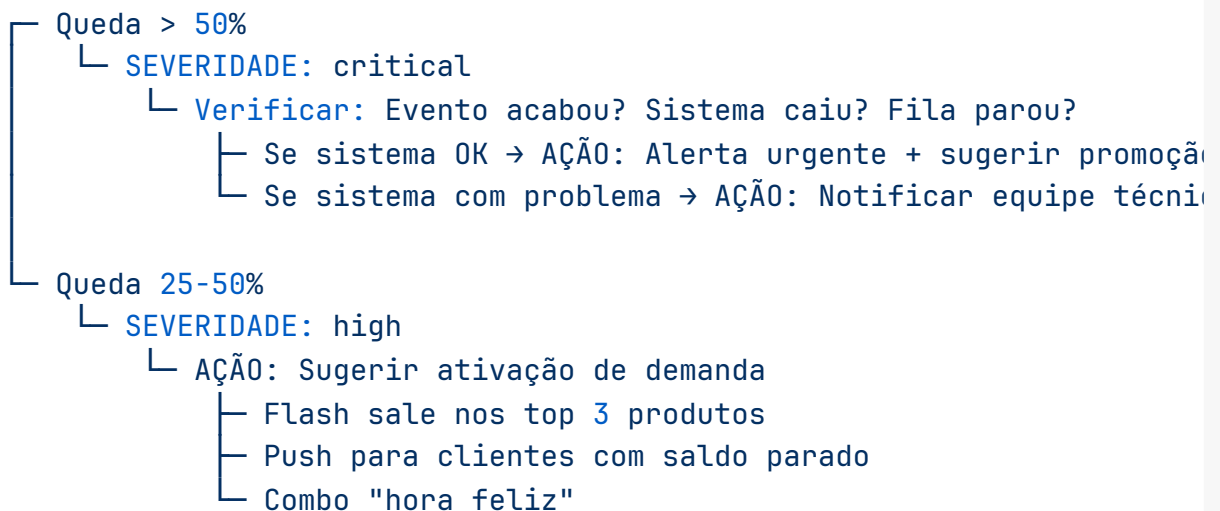
TRIGGER: `minutesRemaining < threshold`



Implementação: `generateStockForecast()` calcula `minutesRemaining = currentStock / consumptionPerHour × 60`. Resultado alimenta `generateProactivePredictions()`.

4. Queda de Receita

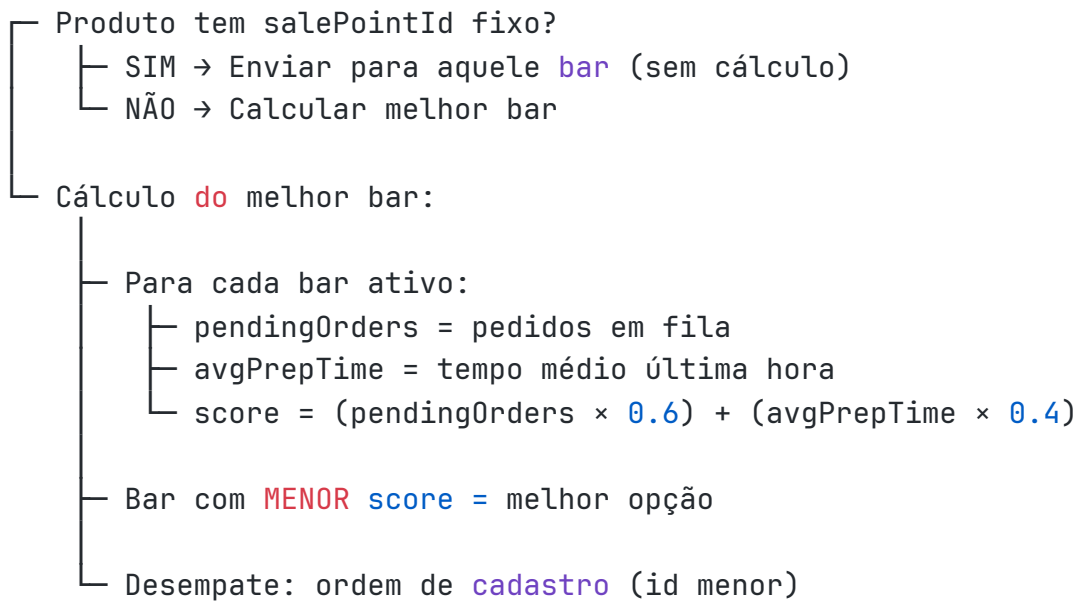
TRIGGER: `revenuePerMinute < baseline × 0.75`



Implementação: `detectAnomaliesRules()` Rule 4 → `generateSuggestions()` com contexto de `unusedBalance` e `topProducts`.

5. Smart Routing (Distribuição de Pedidos)

TRIGGER: Novo pedido criado



Implementação: `server/dispatch.ts` — `dispatchOrder()` retorna `DispatchResult` com `salePointId`, `reason`, e lista de `candidates` com scores.

Extensões planejadas (V2):

- GPS/proximidade do cliente (peso configurável)
 - Preferência manual do produtor (override)
 - VIP routing (prioridade por segmento)
-

6. Pico de Demanda

TRIGGER: `ordersPerHour > baseline * 1.5`

- └ Spike > 80%
 - └ URGÊNCIA: high
 - └ AÇÃO: "Reforçar equipe nos pontos de maior fluxo"
 - └ Impacto: +R\$ (revenuePerMinute * 30)
- └ Spike 50-80%
 - └ URGÊNCIA: medium
 - └ AÇÃO: Monitorar + preparar reforço
 - └ Verificar estoque dos top 5 produtos

Implementação: `generateProactivePredictions()` tipo `demand_spike`.

7. Detecção de Fraude

TRIGGER: `ordersPerHour > baseline * 3` (com `baseline > 5`)

OU

TRIGGER: `avgTicket > baseline * 2`

- └ Transaction burst (volume anômalo)
 - └ SEVERIDADE: high/critical
 - └ AÇÃO: Alertar + sugerir verificação manual
 - └ "Verificar possível automação/fraude"
- └ Ticket spike (valor anômalo)
 - └ SEVERIDADE: high/critical
 - └ AÇÃO: Alertar + verificar últimas transações
 - └ "Verificar possível manipulação de valores"

Implementação: `detectAnomaliesRules()` Rules 7 e 8.

8. Previsão Climática (Planejamento)

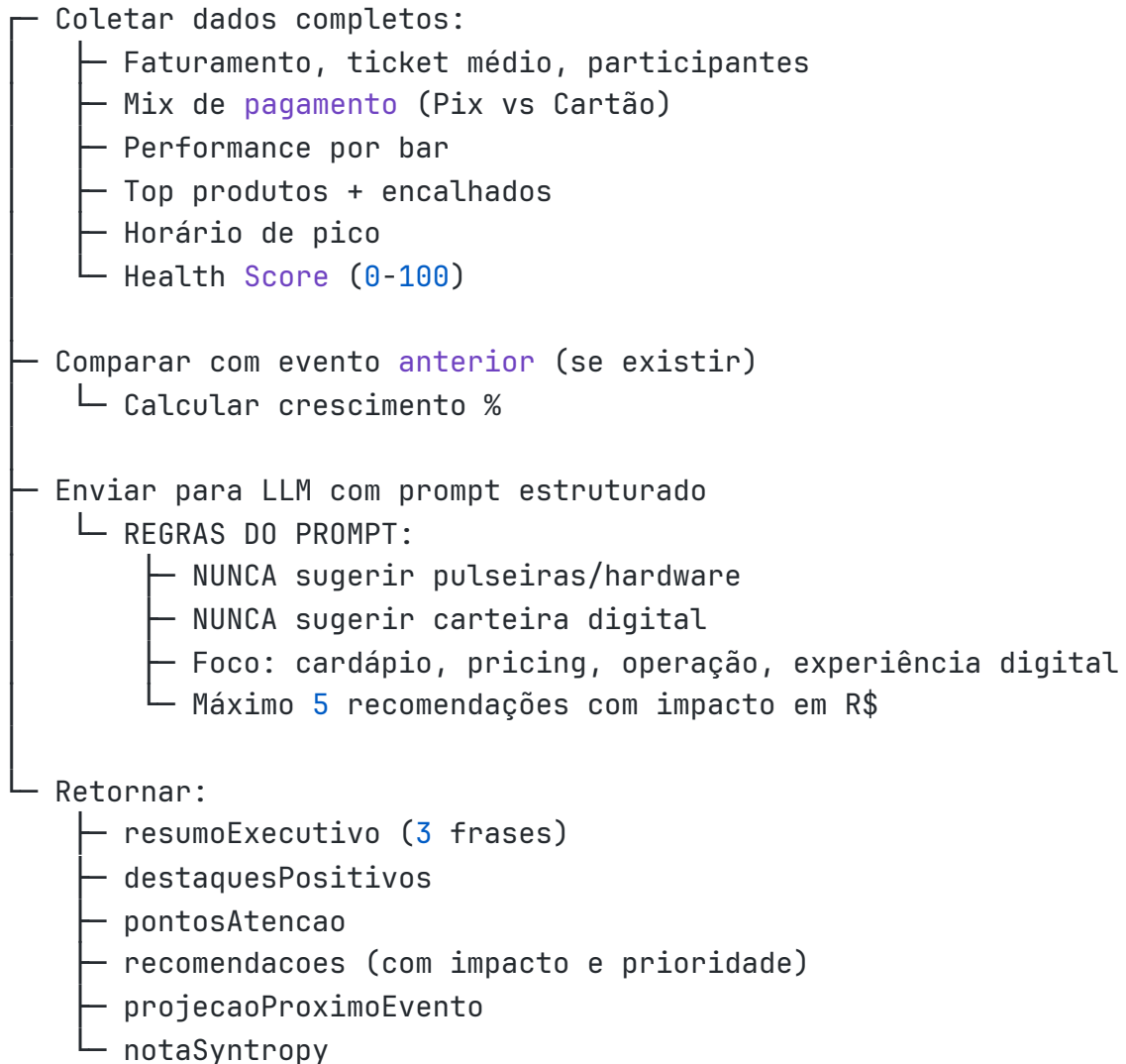
TRIGGER: Evento tem data + localização definidos

- └ Consultar Open-Meteo forecast para data/local
- └ Temperatura > 32°C
 - └ AJUSTE: +30% estoque de bebidas geladas
 - └ Histórico confirma? (se sim, confiança alta)
- └ Chuva prevista > 5mm
 - └ AJUSTE: -20% público estimado
 - └ Se evento coberto → ignorar
- └ Temperatura < 18°C
 - └ AJUSTE: +20% drinks quentes, -15% cerveja
 - └ Sugerir: "Adicionar opções quentes ao cardápio"

Estado atual: ✗ Não implementado no planejamento. Open-Meteo está integrado apenas no pós-evento (análise histórica). **Próximo passo:** Integrar forecast API no syntropyPlanner.

9. Relatório Pós-Evento (Vision)

TRIGGER: Evento `encerrado` (status = "finished")



Implementação: `postEventReport.recommendations` — completo e funcional.

Pipeline de Análise (Orquestrador)

O ciclo completo de decisão durante um evento segue 5 camadas sequenciais:

CAMADA 1 – COLETA (collectEventSnapshot)

Agrega todos os dados do evento em um snapshot

Frequência: a cada chamada manual ou scheduled

CAMADA 2 – BASELINE (calculateBaselines)

Calcula médias móveis dos últimos 30 minutos

Serve como referência para detectar desvios

CAMADA 3 – DETECÇÃO (detectAnomalies)

Regras determinísticas (8 rules) + LLM (padrões ocultos)

Cada anomalia tem: tipo, severidade, métrica, desvio %

CAMADA 4 – SUGESTÃO (generateSuggestions)

LLM gera ações práticas para cada anomalia

Cada sugestão tem: tipo, impacto em R\$, prioridade

CAMADA 5 – EXECUÇÃO (executeAction)

Produtor aprova → sistema executa → registra resultado

Tipos: create_promo, restock, adjust_price, notify_team

Regras Determinísticas (8 Rules)

#	Nome	Trigger	Severidade
1	Queue time	$\text{avgServiceTime} > \text{baseline} \times 1.5$	high/critical
2	Orders deviation	$\text{ordersPerHour} \text{ desvio} > 30\%$	medium/high
3	Low stock	$\text{currentStock} < \text{threshold}$	medium/high/critical
4	Revenue drop	$\text{revenuePerMinute} < \text{baseline} \times 0.75$	high/critical
5	High unused balance	$\text{unusedBalance} > \text{totalRevenue} \times 0.4$	medium
6	Underperforming salepoint	$\text{revenue} < \text{média} \times 0.4$	medium
7	Transaction burst (fraude)	$\text{ordersPerHour} > \text{baseline} \times 3$	high/critical
8	Ticket spike (fraude)	$\text{avgTicket} > \text{baseline} \times 2$	high/critical

Tipos de Ação Executável

Tipo	O que faz	Reversível?
<code>create_promo</code>	Cria promoção com desconto por 30min	Sim (desativar promo)
<code>flash_sale</code>	Promoção relâmpago	Sim
<code>create_combo</code>	Cria combo de produtos	Sim
<code>restock</code>	Alerta de reabastecimento	N/A (informativo)
<code>adjust_price</code>	Registra sugestão de ajuste	N/A (informativo)
<code>reallocate_staff</code>	Notifica equipe para realocar	N/A (informativo)
<code>notify_team</code>	Envia mensagem para equipe	N/A (informativo)

Evolução Planejada

Versão	Capacidade	Status
V1 (atual)	Regras + LLM reativo	✔ Implementado
V2	Promoções sugeridas proativamente	◆ Infra pronta, falta trigger automático
V3	Precificação dinâmica	✗ Não implementado
V4	Decisões autônomas (Nível 3 completo)	✗ Não implementado
V5	Aprendizado cross-produtor	✗ Não implementado

Para Desenvolvedores

Onde vive a lógica de decisão:

Componente	Arquivo	Responsabilidade
Snapshot	<code>server/intelligence-engine.ts:70</code>	Coleta dados
Baselines	<code>server/intelligence-engine.ts:166</code>	Calcula referências
Rules	<code>server/intelligence-engine.ts:253</code>	8 regras determinísticas
LLM Detection	<code>server/intelligence-engine.ts:376</code>	Padrões ocultos
Suggestions	<code>server/intelligence-engine.ts:501</code>	Gera ações
Execution	<code>server/intelligence-engine.ts:714</code>	Executa ações aprovadas
Dispatch	<code>server/dispatch.ts</code>	Smart Routing
Predictions	<code>server/intelligence-engine.ts:955</code>	Previsões proativas

Regra para novas decisões: Toda nova árvore de decisão deve seguir o padrão:

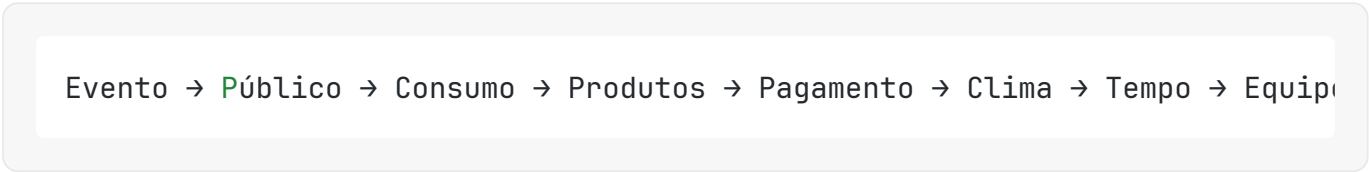
1. Definir TRIGGER (condição mensurável)
2. Definir VERIFICAÇÃO (contexto adicional)
3. Definir AÇÃO (com tipo e impacto estimado)
4. Definir REVERSÃO (como desfazer)
5. Documentar aqui antes de implementar

Knowledge Graph — O que a Fluxa Sabe

Este documento mapeia todo o conhecimento que a Fluxa acumula. Cada dado coletado alimenta a Syntropy e torna a plataforma mais inteligente.

Visão Geral

A Fluxa não é apenas um sistema de pedidos. Ela é um **organismo que aprende**. Cada interação gera dados. Cada dado alimenta a Syntropy. Cada decisão da Syntropy gera mais dados.



O que a Fluxa sabe sobre o EVENTO

Dado	Fonte	Uso
Tipo (festival, festa, corporativo)	Onboarding	Previsão de comportamento
Tamanho esperado	Produtor + ingressos	Dimensionamento
Local (indoor, outdoor, cidade)	Configuração	Logística + clima
Data e horário	Configuração	Sazonalidade
Duração	Configuração	Estimativa de consumo
Histórico de eventos similares	Plataforma	Benchmark

O que a Fluxa sabe sobre o PÚBLICO

Dado	Fonte	Uso
Quantidade real vs esperada	Ingressos + check-in	Calibração de previsão
Perfil demográfico	Dados de compra	Personalização
Horário de chegada	Check-in	Previsão de pico
Padrão de consumo	Pedidos	Sugestão de cardápio
Ticket médio	Pedidos	Precificação
Frequência (recorrente?)	CRM	Fidelidade
Preferências	Histórico	Recomendação

O que a Fluxa sabe sobre o CONSUMO

Dado	Fonte	Uso
O que comprou	Pedidos	Ranking de produtos
Quando comprou	Timestamp	Padrão temporal
Onde comprou (qual bar)	Ponto de venda	Distribuição
Quanto gastou	Pagamento	Ticket médio
Combinações frequentes	Pedidos	Sugestão de combos
Horário de pico	Pedidos	Alocação de equipe
Tempo entre pedidos	Timestamps	Engajamento

O que a Fluxa sabe sobre os PRODUTOS

Dado	Fonte	Uso
Mais vendidos	Pedidos	Destaque no cardápio
Mais lucrativos (margem)	Preço - custo	Recomendação
Menos vendidos	Pedidos	Promoção ou remoção
Elasticidade de preço	Promoções	Precificação dinâmica
Tempo de preparo	KDS	Smart Routing
Taxa de ruptura	Estoque	Previsão de compra
Sazonalidade	Histórico	Planejamento

O que a Fluxa sabe sobre PAGAMENTO

Dado	Fonte	Uso
Mix Pix vs Cartão	Transações	Custo de processamento
Tempo médio de pagamento	Transações	Otimização de fila
Taxa de abandono	Pedidos cancelados	UX
Valor médio por transação	Transações	Precificação
Horário de maior volume	Transações	Capacidade

O que a Fluxa sabe sobre o CLIMA

Dado	Fonte	Uso
Temperatura	API meteorológica	Previsão de consumo
Chuva	API meteorológica	Ajuste de público
Umidade	API meteorológica	Tipo de bebida
Correlação clima × vendas	Histórico	Forecast

O que a Fluxa sabe sobre o TEMPO

Dado	Fonte	Uso
Horário de pico	Pedidos	Alocação de equipe
Horário de baixa	Pedidos	Promoções
Duração média de permanência	Check-in/out	Planejamento
Padrão por dia da semana	Histórico	Sazonalidade
Padrão por mês	Histórico	Planejamento anual

O que a Fluxa sabe sobre a EQUIPE

Dado	Fonte	Uso
Velocidade de preparo	KDS	Eficiência
Pedidos por operador	KDS	Produtividade
Erros/cancelamentos	Pedidos	Treinamento
Horários de pico por bar	Pedidos	Escala
Número ideal por bar	Histórico	Dimensionamento

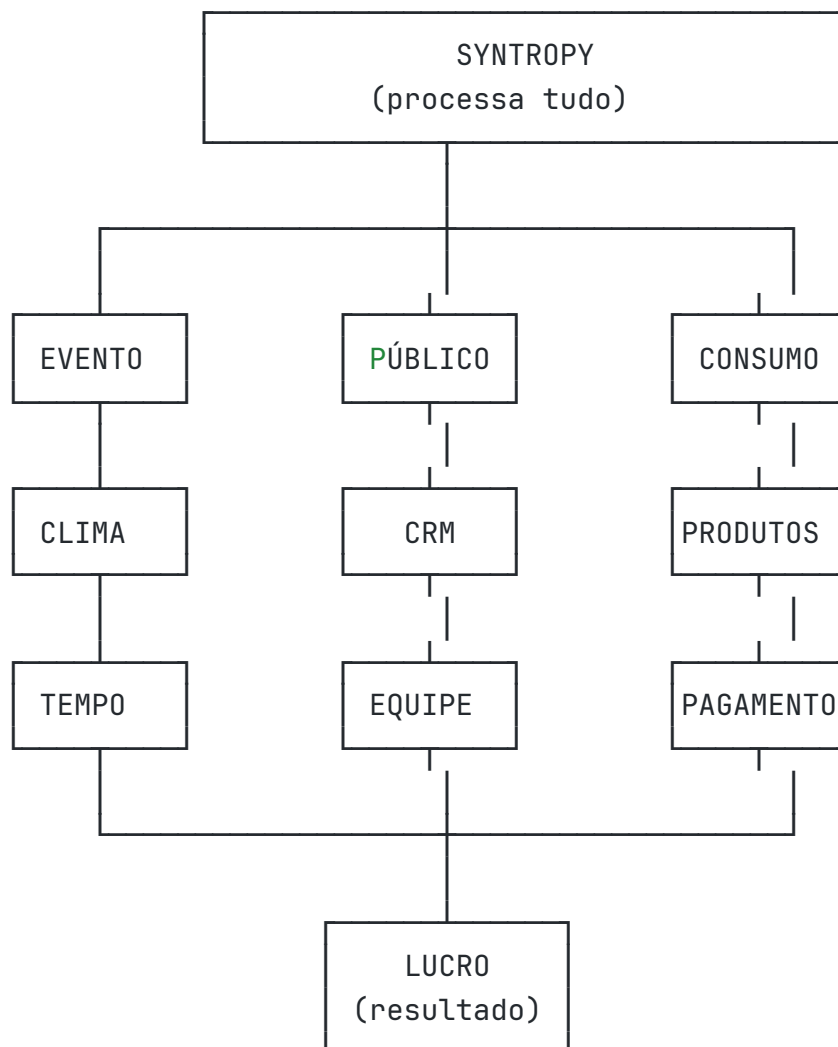
O que a Fluxa sabe sobre FORNECEDORES (futuro)

Dado	Fonte	Uso
Preço por produto	Marketplace	Comparação
Prazo de entrega	Marketplace	Planejamento
Qualidade (avaliação)	Produtores	Recomendação
Disponibilidade	Marketplace	Alternativas

O que a Fluxa sabe sobre o LUCRO

Dado	Fonte	Uso
Faturamento bruto	Transações	Relatório
Custos operacionais	Estoque + equipe	Margem
Margem por produto	Preço - custo	Otimização
Margem por evento	Consolidado	Benchmark
Evolução histórica	Eventos anteriores	Tendência
Projeção futura	Forecast	Planejamento

Como tudo se conecta



Por que isso importa

Cada dado isolado é inútil. Mas quando a Syntropy cruza **todos esses dados simultaneamente**, ela consegue:

1. **Prever** o que vai acontecer antes de acontecer
2. **Recomendar** ações que aumentam lucro
3. **Aprender** padrões que humanos não enxergam
4. **Comparar** eventos entre si (benchmark)
5. **Evoluir** continuamente sem intervenção humana

Privacidade e LGPD

- Dados individuais pertencem ao produtor
 - Aprendizado coletivo usa dados anonimizados e agregados
 - Nenhum produtor vê dados de outro produtor
 - Participantes podem solicitar exclusão (LGPD)
 - Dados de pagamento são processados por MP/Stripe (PCI-compliant)
-

Syntropy Learning Engine

Como a Syntropy aprende e evolui.

Este documento descreve os mecanismos de aprendizado da Syntropy — como ela acumula inteligência a cada evento, a cada produtor, e eventualmente a cada rede.

Filosofia de Aprendizado

A Syntropy não é treinada uma vez e congelada. Ela aprende **continuamente** em 3 dimensões:

Dimensão	O que aprende	Quando aprende	Benefício
Evento	Padrões daquele evento específico	Durante e após o evento	Decisões melhores em tempo real
Produtor	Padrões do produtor ao longo do tempo	Após cada evento	Planejamento mais preciso
Rede	Padrões do mercado de eventos	Agregação anônima	Benchmark e melhores práticas

Camada 1 — Aprendizado por Evento (Implementado)

O que acontece durante o evento

A cada ciclo de análise (chamada ao intelligence-engine), a Syntropy:

1. **Coleta snapshot** — agrega todos os dados do evento naquele instante
2. **Calcula baselines** — médias móveis dos últimos 30 minutos
3. **Compara** — desvio atual vs baseline
4. **Ajusta sensibilidade** — se o evento é naturalmente volátil (festival) vs estável (corporativo)

Evento começa

- Primeiros 30min: Syntropy OBSERVA (sem baseline)
 - └ Coleta dados para formar a primeira referência
- Após 30min: Baseline formado
 - └ Começa a detectar anomalias
- A cada ciclo: Baseline se atualiza (janela deslizante)
 - └ Adapta-se ao ritmo natural do evento
- Evento encerra: Dados consolidados para Vision

Dados que persistem após o evento

Dado	Tabela	Uso futuro
Faturamento total	events	Projeção do próximo evento
Ticket médio	events	Sugestão de pricing
Mix de pagamento	Calculado dos orders	Planejamento de taxas
Top produtos	order_items	Sugestão de cardápio
Produtos encalhados	order_items + ingredients	Ajuste de estoque
Performance por bar	orders.salePointId	Layout operacional
Horário de pico	orders.createdAt	Escala de equipe
Health Score	postEventReports	Evolução do produtor

Camada 2 — Aprendizado por Produtor (Parcialmente Implementado)

O que a Syntropy sabe sobre cada produtor

Após N eventos, a Syntropy constrói um **perfil operacional** do produtor:

PERFIL DO PRODUTOR (acumulado)

Tipo predominante: Festival / Corporativo / Bar / Festa

Público médio: X pessoas

Ticket médio histórico: R\$ Y

Mix de pagamento: Z% Pix, W% Cartão

Produtos campeões: [lista]

Produtos que sempre encaixam: [lista]

Horário de pico típico: HH:MM - HH:MM

Taxa de conversão (presentes → compradores): X%

Health Score médio: N/100

Tendência: melhorando / estável / piorando

Como isso melhora o planejamento

Quando o produtor cria um novo evento, a Syntropy usa o perfil para:

Entrada	Sem histórico (1º evento)	Com histórico (3+ eventos)
Público estimado	Produtor informa manualmente	Syntropy sugere baseado em histórico
Estoque sugerido	Fórmula genérica	Baseado em consumo real anterior
Equipe necessária	Estimativa padrão	Baseado em performance anterior
Cardápio	Template genérico	Top produtos do produtor
Preço sugerido	Média de mercado	Ticket médio real + margem

Estado atual: O Vision (relatório pós-evento) já gera recomendações comparando com evento anterior. O Planner (syntropyPlanner) usa dados do evento anterior quando disponível. **Falta:** Perfil consolidado multi-evento com tendências.

Camada 3 — Aprendizado de Rede (Não Implementado)

Visão futura: inteligência coletiva

Quando a Fluxa tiver 50+ produtores ativos, a Syntropy poderá:

APRENDIZADO DE REDE (futuro)

Benchmark:

"Seu ticket médio está 20% abaixo de eventos similares"

Melhores práticas:

"Produtores com seu perfil vendem 30% mais com combos"

Alertas de mercado:

"Preço de cerveja subiu 15% – ajustar cardápio?"

Sazonalidade:

"Eventos em dezembro faturam 40% mais – preparar estoque"

Princípios de privacidade para rede

Regra	Descrição
Anonimização	Dados agregados nunca identificam o produtor
Opt-in	Produtor escolhe participar do benchmark
Benefício mútuo	Só recebe insights de rede quem contribui
Segmentação	Comparação apenas entre eventos similares

Mecanismo de Feedback Loop

A Syntropy aprende não apenas com dados, mas com as **decisões do produtor**:

Syntropy sugere ação

- Produtor APROVA → executa → resultado registrado
 - └ Feedback: "Essa sugestão funcionou" (reforço positivo)
- Produtor REJEITA → motivo registrado (opcional)
 - └ Feedback: "Essa sugestão não era boa" (ajuste)
- Produtor IGNORA → **timeout**
 - └ Feedback: "Sugestão irrelevante" (reduzir prioridade)

Estado atual: aiInsights registra status (pending/applied/dismissed) e appliedAt / dismissedAt . O feedback existe na estrutura mas ainda não alimenta um modelo de refinamento.

Evolução planejada do feedback

Fase	Mecanismo	Resultado
V1 (atual)	Registro de aceite/rejeição	Histórico auditável
V2	Score de acurácia por tipo de sugestão	Priorizar sugestões com alto aceite
V3	Ajuste de thresholds por produtor	Menos falsos positivos
V4	Modelo de preferência do produtor	Sugestões personalizadas

Dados de Treinamento Disponíveis

O que a Fluxa já coleta e pode usar para aprendizado:

Fonte	Volume por evento	Uso potencial
Pedidos (orders)	200-5000 registros	Padrões de consumo
Itens (order_items)	500-15000 registros	Preferências de produto
Ingredientes (ingredients)	20-100 registros	Otimização de estoque
Transações financeiras	200-5000 registros	Padrões de pagamento
Insights gerados (aiInsights)	5-50 por evento	Eficácia da IA
Ações executadas (iaActions)	2-20 por evento	Feedback de decisão
Relatórios pós-evento	1 por evento	Consolidação
Contratos (contracts)	1 por evento	Contexto financeiro

O Flywheel do Aprendizado

Mais eventos na Fluxa

- Mais dados coletados

- Melhores previsões

- Menos desperdício

- Mais faturamento

- Produtor faz mais eventos na Fluxa

- (volta ao início)

- Benchmark de rede mais preciso

- Novos produtores atraídos pelo benchmark

- (volta ao início)

- Custo de saída aumenta

- Produtor perde todo o histórico se sair

- Lock-in natural (sem contrato)

Para Desenvolvedores

Onde vive o aprendizado:

Componente	Arquivo	O que faz
Baselines (intra-evento)	<code>intelligence-engine.ts:166</code>	Médias móveis 30min
Post-event consolidation	<code>intelligence-engine.ts</code> (<code>postEventReport</code>)	Consolida dados finais
Feedback tracking	<code>aiInsights</code> table	Registra aceite/rejeição
Action logging	<code>iaActions</code> table	Registra resultado de ações
Producer profile	✗ Não existe ainda	Perfil consolidado multi-evento
Network learning	✗ Não existe ainda	Agregação cross-produtor

Próximos passos de implementação:

1. Criar tabela `producer_profiles` com métricas acumuladas
 2. Trigger pós-evento que atualiza o perfil
 3. Planner consultar perfil ao gerar sugestões
 4. Score de acurácia por tipo de insight
 5. Dashboard “Evolução” mostrando tendência do produtor
-

Syntropy Inference Engine

Como a Syntropy transforma dados em recomendações.

Este documento descreve o processo de inferência — a ponte entre “dados brutos” e “ação prática”. Não é sobre o que ela sabe (Knowledge Graph) nem como decide (Decision Engine), mas sobre **como ela raciocina para chegar a uma conclusão**.

O que é Inferência na Syntropy

Inferência é o processo de:

DADOS BRUTOS → CONTEXTO → PADRÃO → CONCLUSÃO → RECOMENDAÇÃO

A Syntropy não “adivinha”. Ela segue um pipeline lógico onde cada etapa é rastreável e explicável.

Pipeline de Inferência

Visão geral das 5 etapas

ETAPA 1 – NORMALIZAÇÃO Dados brutos → Métricas comparáveis Ex: 847 pedidos em 4h → 211 pedidos/hora
ETAPA 2 – CONTEXTUALIZAÇÃO Métrica isolada → Métrica com referência Ex: 211/h é alto? Baseline era 180/h → +17% (normal)
ETAPA 3 – CORRELAÇÃO Múltiplas métricas → Padrão identificado Ex: vendas +17% + estoque caindo + fila subindo = PICO
ETAPA 4 – PROJEÇÃO Padrão atual → Cenário futuro Ex: Se continuar assim, cerveja acaba em 45min
ETAPA 5 – PRESCRIÇÃO Cenário futuro → Ação recomendada com impacto Ex: "Repor 200 cervejas agora. Evita perda de R\$ 3.200"

Etapa 1 — Normalização

O que faz

Transforma dados brutos heterogêneos em métricas padronizadas e comparáveis.

Exemplos de normalização

Dado bruto	Métrica normalizada
847 pedidos em 4h03min	209 pedidos/hora
R\$ 63.500 faturamento	R\$ 264/min
12 pedidos pendentes no Bar 1	4 pedidos/operador
180 cervejas consumidas em 3h	60 un/hora
420 participantes, 312 compraram	74% taxa de conversão

Implementação atual

O `collectEventSnapshot()` já faz essa normalização:

- `ordersPerHour` = total de pedidos / horas decorridas
- `revenuePerMinute` = faturamento / minutos decorridos
- `avgTicket` = faturamento / número de pedidos
- `consumptionPerHour` = unidades consumidas / horas (por ingrediente)

Etapa 2 — Contextualização

O que faz

Dá significado à métrica comparando com uma referência. Sem contexto, “211 pedidos/hora” não significa nada.

Fontes de contexto (em ordem de prioridade)

Fonte	Quando disponível	Confiança
Baseline intra-evento (30min)	Após 30min de evento	Alta
Evento anterior do produtor	2º evento em diante	Média-alta
Perfil do produtor	3+ eventos	Média
Benchmark de rede	50+ produtores	Média-baixa
Estimativa genérica	Sempre	Baixa

Lógica de contextualização

Para cada métrica:

- Tem baseline intra-evento?
 - └ SIM → Comparar (desvio %)
 - └ Desvio < 20% → NORMAL
 - └ Desvio 20-50% → ATENÇÃO
 - └ Desvio > 50% → ANOMALIA
- Não tem baseline? Tem evento anterior?
 - └ SIM → Comparar com mesmo horário **do** evento passado
- Não tem nada?
 - └ Usar estimativa genérica (ex: 150 pedidos/h para 500 pessoas)

Implementação atual

`calculateBaselines()` usa janela de 30 minutos. `getLatestBaselines()` recupera a última baseline salva. Comparação com evento anterior existe apenas no Vision (pós-evento), não em tempo real.

Etapa 3 — Correlação

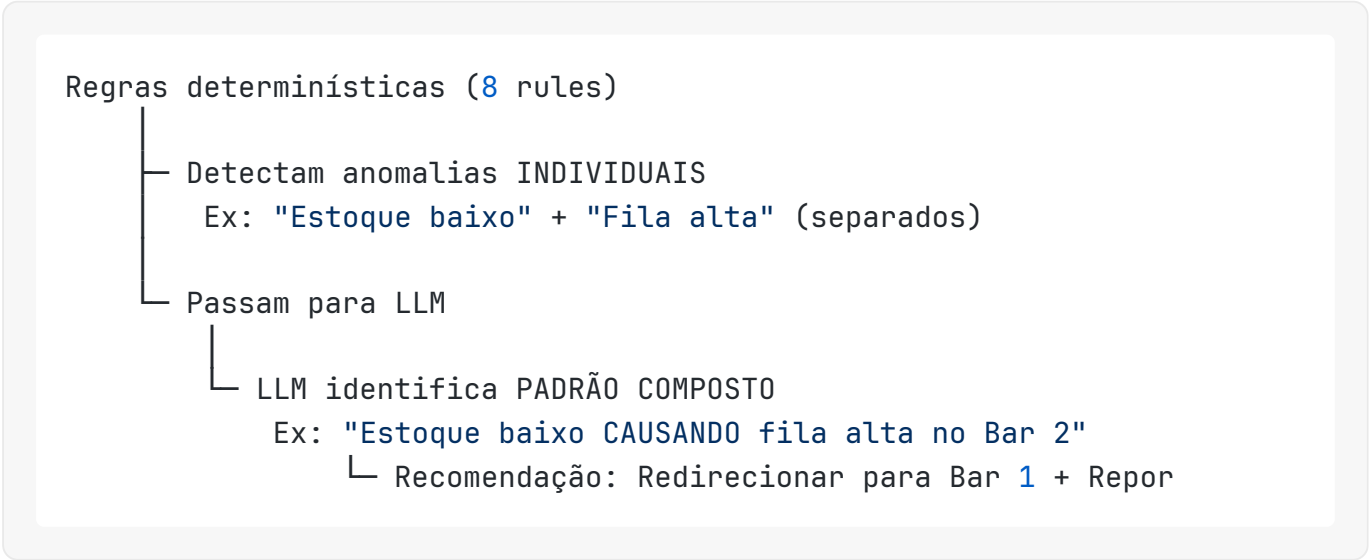
O que faz

Cruza múltiplas métricas para identificar um **padrão composto** que nenhuma métrica isolada revelaria.

Padrões que a Syntropy reconhece

Padrão	Sinais combinados	Significado
Pico de demanda	Pedidos ↑ + Fila ↑ + Estoque ↓	Evento aquecendo, preparar reforço
Desaceleração	Pedidos ↓ + Revenue ↓ + Saldo parado ↑	Público perdendo interesse
Gargalo operacional	Pedidos estáveis + Fila ↑ + 1 bar lento	Problema localizado
Produto viral	1 produto ↑↑ + outros ↓	Concentração de demanda
Fraude	Transações ↑↑↑ + Ticket ↑↑ + Padrão irregular	Possível manipulação
Fim natural	Pedidos ↓ gradual + Saldo ↓ + Sem anomalia	Evento acabando normalmente

Como a correlação funciona



Implementação atual

As 8 regras detectam anomalias individuais. O `detectAnomaliesLLM()` recebe o snapshot completo e identifica padrões que as regras não pegam. O `generateSuggestions()` recebe ambos e gera ações contextualizadas.

Etapa 4 — Projeção

O que faz

Extrapolando o padrão atual para prever o que vai acontecer se nada mudar.

Tipos de projeção

Tipo	Cálculo	Exemplo
Estoque	<code>currentStock / consumptionPerHour</code>	“Cerveja acaba em 45min”
Receita	<code>revenuePerMinute × minutosRestantes</code>	“Projeção: R\$ 127.500 no fim”
Fila	tendência dos últimos 15min	“Fila chega a 20min em 30min”
Ruptura	item com consumo > reposição	“3 itens esgotam antes das 23h”

Implementação atual

`generateStockForecast()` calcula `minutesRemaining` para cada ingrediente. `generateProactivePredictions()` projeta cenários de estoque, demanda e receita. O Mission Control mostra “Receita Projetada” baseada em tendência.

Projeções futuras (não implementadas)

Projeção	Dados necessários	Impacto
Clima → Consumo	Open-Meteo + histórico	Ajuste de estoque pré-evento
Horário → Pico	Histórico do produtor	Escala de equipe otimizada
Dia da semana → Público	Dados de rede	Previsão de receita
Preço → Elasticidade	Histórico de vendas	Precificação dinâmica

Etapa 5 — Prescrição

O que faz

Transforma a projeção em uma **ação concreta** com impacto estimado em R\$.

Anatomia de uma recomendação

Toda recomendação da Syntropy segue este formato:

TÍTULO: "Repor cerveja agora"

CONTEXTO: "Consumo: 60 un/h. Estoque: 45 un."

PROJEÇÃO: "Esgota em 45 minutos"

AÇÃO: "Repor 120 unidades"

IMPACTO: "Evita perda de R\$ 3.200 em vendas"

URGÊNCIA: critical / high / medium / low

TEMPO PARA AGIR: 45 minutos

BOTÃO: [Executar] [Ajustar] [Ignorar]

Como o impacto é calculado

Tipo de ação	Fórmula de impacto
Restock	$\text{unidades_em_risco} \times \text{preço_médio_unitário}$
Promoção	$\text{estoque_parado} \times \text{preço} \times \text{desconto} \times \text{taxa_conversão_estimada}$
Realocação	$\text{pedidos_perdidos_estimados} \times \text{ticket_médio}$
Flash sale	$(\text{público_com_saldo} \times \text{valor_médio_saldo} \times 0.3)$

Implementação atual

`generateSuggestions()` retorna array de `SuggestedAction` com `estimatedImpact` em R\$. O LLM calcula o impacto baseado nos dados do snapshot. `executeAction()` implementa a ação quando aprovada.

Dois Motores de Inferência

A Syntropy usa dois motores complementares:

Motor 1 — Determinístico (Rules)

Vantagens:

- ✓ Rápido (< 10ms)
- ✓ Previsível
- ✓ Auditável
- ✓ Funciona offline

Limitações:

- × Só pega padrões conhecidos
- × Não correlaciona
- × Não explica "porquê"
- × Falsos positivos em eventos atípicos

Motor 2 — LLM (Generativo)

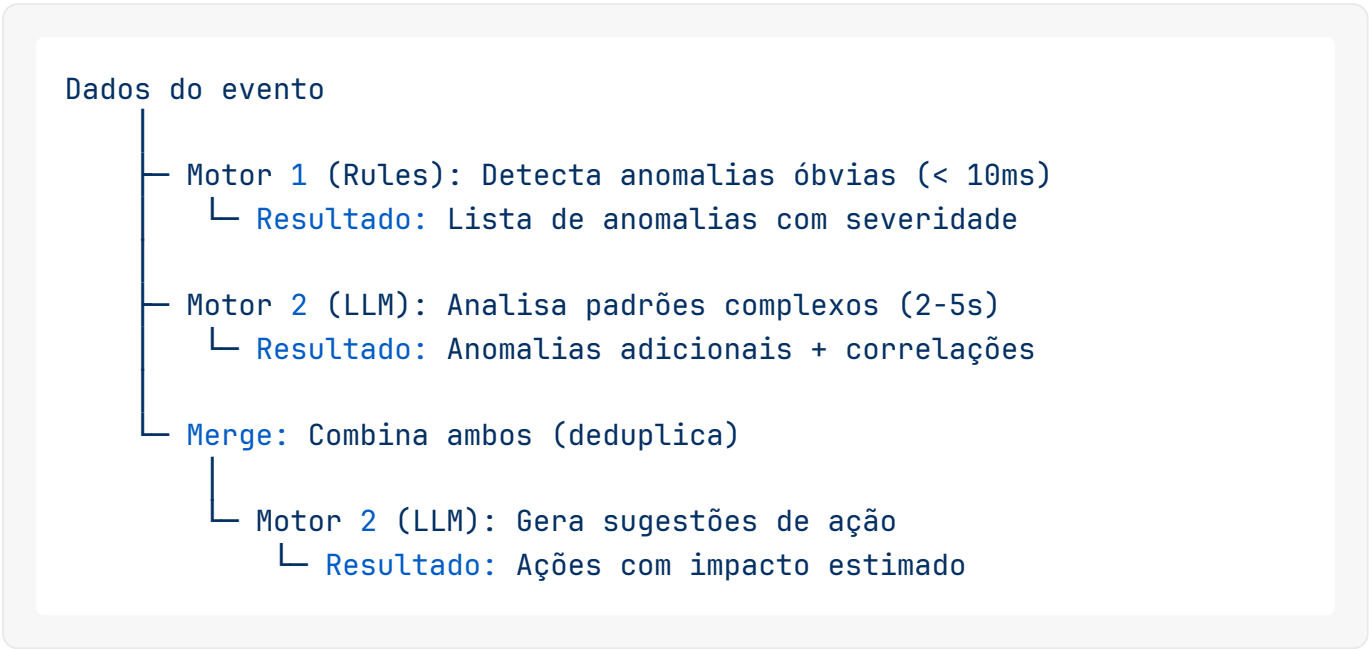
Vantagens:

- ✓ Encontra padrões ocultos
- ✓ Correlaciona múltiplos
- ✓ Explica em linguagem natural
- ✓ Adapta-se ao contexto

Limitações:

- × Lento (2-5s)
- × Custo por chamada
- × Pode "alucinar"
- × Depende de internet

Como trabalham juntos



Fallback: Se o LLM falhar (timeout, erro, offline), o sistema funciona apenas com regras determinísticas. Nunca fica cego.

Qualidade da Inferência

Métricas de qualidade (a implementar)

Métrica	Definição	Meta
Precision	% de alertas que eram reais	> 80%
Recall	% de problemas reais detectados	> 70%
Acceptance rate	% de sugestões aceitas pelo produtor	> 50%
Impact accuracy	Impacto real vs estimado	±30%
Time to action	Tempo entre sugestão e execução	< 5min

Como melhorar a qualidade

Fase 1 (atual): Thresholds fixos

- └ Problema: Muitos falsos positivos em eventos pequenos

Fase 2 (próxima): Thresholds adaptativos

- └ Ajusta por tipo de evento e histórico do produtor

Fase 3 (futuro): Modelo treinado

- └ Usa feedback (aceite/rejeição) para calibrar

Fase 4 (visão): Auto-calibração

- └ Syntropy ajusta seus próprios parâmetros

Exemplos Completos de Inferência

Exemplo 1: “Cerveja vai acabar”

NORMALIZAÇÃO:

- Cervejas consumidas: 180 em 3h = 60/hora
- Estoque atual: 45 unidades

CONTEXTUALIZAÇÃO:

- Baseline: 50/hora (evento está acima do normal)
- Evento anterior: 55/hora no mesmo horário

CORRELAÇÃO:

- Temperatura: 34°C (correlação positiva com cerveja)
- Público crescendo (mais pessoas chegando)
- Outros drinks estáveis (demanda concentrada em cerveja)

PROJEÇÃO:

- Taxa atual: 60/hora (pode subir para 70 com temperatura)
- Estoque: 45 unidades
- Tempo até esgotamento: ~40 minutos

PRESCRIÇÃO:

- Ação: Repor 120 unidades agora
- Impacto: Evita perda de R\$ 3.600 (60 cervejas × R\$ 18 × 3.3 margem)
- Urgência: CRITICAL
- Tempo para agir: 40 minutos

Exemplo 2: “Bar 2 está lento”

NORMALIZAÇÃO:

- Bar 1: 8 pedidos pendentes, 3min tempo médio
- Bar 2: 22 pedidos pendentes, 9min tempo médio
- Bar 3: 6 pedidos pendentes, 2.5min tempo médio

CONTEXTUALIZAÇÃO:

- Baseline Bar 2: 12 pendentes, 5min (muito acima)
- Bar 2 tem 2 operadores vs 3 nos outros

CORRELAÇÃO:

- Produto mais pedido no Bar 2: Caipirinha (preparo longo)
- Caipirinha disponível em todos os bares
- Clientes não sabem que podem pedir em outro bar

PROJEÇÃO:

- Se continuar: fila do Bar 2 chega a 15min em 20min
- Clientes começam a desistir (taxa de abandono sobe)
- Perda estimada: R\$ 2.400/hora

PRESCRIÇÃO:

- Ação 1: Ativar Smart Routing (redirecionar caipirinhas para Bar 1/3)
- Ação 2: Sugerir realocação de 1 operador para Bar 2
- Impacto combinado: Reduz fila de 9min para ~4min
- Urgência: HIGH

Exemplo 3: “Gin Tônica encalhando”

NORMALIZAÇÃO:

- Gin Tônica: 12 vendas em 3h = 4/hora
- Média dos outros drinks: 25/hora
- Estoque: 80 doses (20h de consumo no ritmo atual)

CONTEXTUALIZAÇÃO:

- Evento anterior: Gin Tônica vendeu 18/hora (muito acima)
- Diferença: evento anterior era corporativo, este é festival
- Preço: R\$ 32 (mais caro que média R\$ 22)

CORRELAÇÃO:

- Público mais jovem neste evento (festival)
- Cerveja e drinks baratos dominando
- Margem do Gin Tônica: 65% (permite desconto)

PROJEÇÃO:

- Sem ação: 80 doses sobram (R\$ 2.560 em estoque parado)
- Com promoção 25% off: estimativa +300% vendas = 12/hora

PRESCRIÇÃO:

- Ação: Flash sale "Gin Tônica R\$ 24" por 1 hora
- Impacto: Recupera ~R\$ 1.200 do estoque parado
- Alternativa: Combo "Gin + Petisco" por R\$ 35
- Urgência: MEDIUM (não é urgente, é oportunidade)

Para Desenvolvedores

Onde vive a inferência:

Etapa	Arquivo	Função
Normalização	<code>intelligence-engine.ts</code>	<code>collectEventSnapshot()</code>
Contextualização	<code>intelligence-engine.ts</code>	<code>calculateBaselines()</code> + <code>getLatestBaselines()</code>
Correlação (rules)	<code>intelligence-engine.ts</code>	<code>detectAnomaliesRules()</code>
Correlação (LLM)	<code>intelligence-engine.ts</code>	<code>detectAnomaliesLLM()</code>
Projeção	<code>intelligence-engine.ts</code>	<code>generateProactivePredictions()</code> + <code>generateStockForecast()</code>
Prescrição	<code>intelligence-engine.ts</code>	<code>generateSuggestions()</code>
Execução	<code>intelligence-engine.ts</code>	<code>executeAction()</code>

Regra para novas inferências: Toda nova capacidade de inferência deve:

1. Definir claramente as 5 etapas (normalização → prescrição)
2. Ter fallback determinístico (funcionar sem LLM)
3. Mostrar o “porquê” ao produtor (transparência)
4. Ter impacto estimável em R\$ (justificativa)
5. Ser reversível (segurança)